2022 IEEE International Conference on
Systems, Man, and Cybernetics (SMC)

Motion Planning and Obstacle Avoidance for Robot Manipulators Using Model Predictive Control-based Reinforcement Learning

Adel Baselizadeh
Department of Informatics
University of Oslo
Oslo, Norway
adelb@ifi.uio.no

Weria Khaksar
Faculty of Science and Technology
Norwegian University of Life Sciences
Ås, Norway
weria.khaksar@nmbu.no

Jim Torresen
Department of Informatics and RITMO
Centre for Interdisciplinary Studies in
Rhythm, Time and Motion
University of Oslo
Oslo, Norway
jimtoer@ifi.uio.no

2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC) | 978-1-6654-5258-8 /22/$31.00 ©2022 EU | DOI: 10.1109/SMC53654.2022.9945504

***Abstract*—This paper presents a Nonlinear Model Predictive Control-based Reinforcement Learning (NMPC-based RL) framework for robot manipulators. The controller is developed to address the motion planning problem for robot manipulators in the presence of obstacles. The proposed control scheme includes a parametrized NMPC structure used as an approximator for the RL framework's value function and action-value function. In the NMPC structure, the cost function, system constraints, and the manipulator's model are parameterized. The Q-Learning algorithm based on the Temporal Difference method adjusts the parameters of the NMPC to increase the closed-loop performance of the whole control scheme. The controller has been applied to a 6-degrees-of-freedom (DoF) model of a robot manipulator, aimed at moving its end-effector to reach the desired pose when static obstacles are in the robot's workspace. Numerical simulations demonstrate that the proposed controller can effectively control the end-effector's pose in such a way as to avoid any collisions between the manipulator and the obstacles. It is shown that the learning capability of the proposed NMPC-based RL framework can enhance the efficiency of the control loop up to 21%.**

Keywords— Robot manipulator, Model predictive control, Reinforcement learning, Obstacle avoidance, Motion planning

I. INTRODUCTION

As the main components of many robotic systems, robot manipulators have always been an important consideration in robotics [1]. Robot manipulators are used in diverse fields, including industrial robots [2], surgical robots [3], and personal robot assistants [4]. Safety considerations should be taken into account when using robot manipulators, especially in environments containing many objects in the manipulator's workspace. As a safety consideration, avoiding collisions between the robot manipulator and the objects is of prime importance. Thus, much of the research in this area has been devoted to the collision avoidance problem [6]. A wide variety of (multi-modal) sensing systems [7] and control algorithms are used to address this problem. When it comes to using control methods, various traditional algorithms [8-10] and modern approaches based on Artificial Intelligence (AI) [11] have been proposed to address the collision avoidance problem for robot manipulators and their motion control in general [12].

Model predictive control (MPC) is among the most important traditional control algorithms capable of solving optimization-based control problems, including constrained control/system variables [13]. In robot manipulators, collision avoidance is generally thought of as an optimization problem, where the goal is to maximize the distance between the manipulator's links and obstacles and satisfy joint positions and rotational speed limits. Hence, MPC has been widely used to address this problem [14-16]. Moreover, MPC-based control algorithms are suitable options for solving problems requiring high levels of safety because of their proven stability. However, there are severe limitations associated with MPC-based methods and, in general, most of the traditional model-based control algorithms. These methods are limited by their dependence on having a precise model of the controlled system since the lack of one makes it difficult to control the system effectively [17]. In contrast to traditional methods, AI-based algorithms, particularly reinforcement learning (RL), can efficiently control systems with complex and nonlinear dynamics due to their model-free structures. However, unlike traditional methods, AI-based control algorithms usually lack proof of stability and have safety and stability challenges [18].

Learning-based control schemes resulting from integrating traditional and AI-based control methods can solve the issues associated with each technique and facilitate a balance between safety/stability and efficiency in control loops [19]. Due to their similar optimization-based structures, RL and MPC can be integrated and create an efficient control scheme. The resulting learning-based controller can control complex dynamical systems and ensure the stability of the control loop, despite the lack of a precise model of the system dynamics. So far, different approaches have been proposed to integrate MPC and RL [20, 21]. One method uses MPC as a function approximator (value function and action-value function) in the RL structure. Deep Neural Networks (DNNs) [22] and Q-tables [23] are commonly used as function approximators in RL frameworks. Using DNNs and Q-tables makes proving the stability of RL frameworks challenging or impossible because of their data-driven structures. However, according to [24], in the case of using an appropriate formulation, MPC can approximate the value/action-value

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

function and the optimal policy in an RL framework and guarantee the stability of the control loop. In this framework, RL's value and action-value functions are approximated according to the MPC scheme's cost function. The outputs of MPC, i.e., control inputs, are also used to determine the optimal policy and actions in the RL framework. The MPC architecture, including the cost function, system constraints, and the dynamic system model, is parametrized in this method. Like RL frameworks based on DDNs and Q-tables, a learning algorithm, e.g., Q-learning [23], is used to tune the parameters of the MPC in the available MPC-based RL scheme. The learning algorithm updates the parameters of the MPC in such a way as to optimize a cost function known as RL reward.

Data-driven MPC approaches have already been applied to solve control problems. The contribution of this work is to formulate a novel data-driven MPC control algorithm to address the safety-efficiency trade-off problem in arm manipulators. Although the available framework has been applied to other control problems [28], this is for the first time it has been used for the obstacle avoidance motion planning of robot manipulators. In this regard, the paper resolves the setpoint control problem for a 6-DoF robot manipulator in the presence of static obstacles using the proposed controller. Simulation results indicate that using the available NMPC-based RL controller accurately controls the position of the end-effector to reach the desired pose by moving the robot manipulator in a collision-free path. It has also been shown that the learning capability of the proposed controller helps to increase the overall performance of the control loop. Thus, the available control strategy is an appropriate solution to compensate for the reduced efficiency of the position control for robot manipulators caused by addressing the collision avoidance problem.

The next parts of the paper are organized as follows: Section II presents the 6-DoF kinematical model of the robot manipulator based on the Denavit-Hartenberg (D-H) method. This model is used as the system's dynamics in the MPC formulation. Section III formulates the NMPC-based RL control scheme and details robot manipulators' obstacle avoidance motion planning. Section IV discusses the simulation details and illustrates the results. Finally, Section V concludes the paper with suggestions for future works.

II. A Kinematic Model for a Robot Manipulator

In this paper, a 6-DOF robot manipulator actuated by six flexible joints is applied. Model parameters, joint actuator displacements, and velocities are chosen according to the UR5 robot [25]. Joint configurations are defined as $q = [q_1, q_2, q_3, q_4, q_5, q_6]$ where q_i is the ith joint's value. The robot's end-effector pose is also defined in Cartesian space as $p = [X, Y, Z, \varphi, \theta, \psi]$ where X, Y, and Z are the end-effector's 3D position and φ, θ, and ψ are its orientation by Euler angles representation.

The mathematical kinematic model of the manipulator has been formulated using the D-H method and explained below. D-H notation provides a standard way to formulate robot manipulators' forward kinematics (FK). FK describes the end-effector's position and orientation as a function of the joint configuration. In order to solve FK equations using the D-H method, a Cartesian frame is assigned to every joint, as shown in Fig. 1. The joints' angles (q_i), and three parameters of each link, including offset (d_i), length (a_i), and rotation angle (α_i), are the D-H parameters used to extract the FK of the manipulator.

As illustrated in Eq. 1, a transformation matrix is used to calculate the end-effector position from the configuration matrix. This matrix transforms each pair of consecutive coordinates attached to the joints.

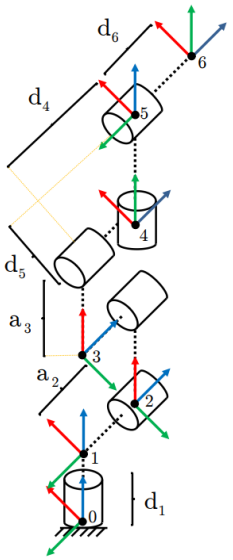


Fig. 1. D-H reference frames assigned to the 6-DoF robot manipulator joints. X, Y, and Z axes are illustrated in red, green, and blue, respectively.

$$T_n^{n-1} = \begin{bmatrix} cos(\theta_n) & -cos(\alpha_n)sin(\theta_n) & sin(\alpha_n)sin(\theta_n) & a_n cos(\theta_n) \\ sin(\theta_n) & cos(\alpha_n)cos(\theta_n) & -sin(\alpha_n)cos(\theta_n) & a_n sin(\theta_n) \\ 0 & sin(\theta_n) & cos(\theta_n) & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

The matrix multiplication illustrated in Eq. 2 finds the relationship between the end-effector's frame and the base frame of the robot as a function of D-H parameters, i.e., q_i, a_i, d_i, and α_i.

$$T_f^0 = \prod_{n=0}^{6} T_n^{n-1} \quad (2)$$

D-H, as described above, solves the manipulator's FK at a position level, that is, it determines the end-effector's pose as a function of the configuration matrix. To represent this relationship by a differential equation that can be used to design the end-effector pose controller, one has to utilize the velocity-level formulation of the FK. The Jacobian matrix (J_A), as illustrated in Eq. 3, is used for this aim.

$$\dot{p} = J_A \dot{q} \quad (3)$$

According to the velocity-level FK model, the end-effector velocity is determined by the rotational velocities of the joints. The controller presented in this paper uses this model for the motion control of the end-effector. The NMPC-based RL controller determines the rotational velocities of joints, which are applied as control actions to the robot manipulator. The formulation of the control algorithm is explained in section III.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

III. MPC-BASED RL CONTROL SCHEME

This section describes the NMPC-based RL control scheme. This controller is used for the motion planning of a 6-DoF robot manipulator in the presence of static obstacles. The goal is to move the end-effector from an arbitrary starting pose, reaching the desired pose without encountering any obstacles on the way. The control scheme consists of an RL structure based on NMPC.

MPC, also referred to as receding horizon control, is an optimization-based control method capable of solving problems involving hard state-input constraints. MPC is concerned with predicting the system's future behavior over a finite time horizon and determining the control input to minimize a cost function while meeting system constraints at the same time. In Eq. 4, the NMPC control problem is shown. The cost function in Eq. 4 is formulated based on the tracking errors between the desired and the actual values of state variables. Thus, by optimizing the cost function, NMPC minimizes the tracking errors, resulting in state variables achieving their desired values.

$$\min_{x,u} \quad T(x_N) + \sum_{k=0}^{N-1} S(x_k, u_k) \qquad (4a)$$

$$s.t. \quad x_{k+1} = f(x_k, u_k) \qquad (4b)$$

$$x_k \le a_k \ , \ u_k \le b_k \ , c(x_k, u_k) \le 0 \qquad (4c)$$

where $T(x_N)$ and $S(x_k, u_k)$ are terminal and stage costs. Terminal cost is used to guarantee the stability of the control loop and the stage cost addresses the optimization problem in NMPC. x_k and u_k are state variable and control actions at time step k, respectively. $f(x_k, u_k)$ demonstrates the controlled system dynamics, and Eq. 4c presents system constraints.

RL usually deals with the approximation of value and action-value functions and finds an optimal policy concerning these determined values based on Bellman equations [26]. DNNs and Q-tables are usually used to approximate value and action-value functions. [24] suggests that under certain conditions, a specific formulation of the NMPC structure presented in Eq. 4 can be used as a function approximator in an RL framework instead of the commonly used DNNs and Q-tables. In this framework, NMPC is used to approximate the value and action-value functions of an RL structure that is applied to control a dynamical system. Like RL structures based on DNNs and Q-tables as function approximators, a learning strategy is used to update the NMPC as the control problem is being solved in the available control framework. The learning strategy updates the NMPC parameters to optimize the RL reward function. A simple illustration of the available NMPC-based RL framework used for the setpoint control of the end-effector in a robotic manipulator has been presented in Fig. 2. As it is shown in Fig. 2, NMPC is a part of RL and is updated in real-time.

The NMPC structure used in this novel formulation of the RL framework must be appropriately parametrized, where the parameters are tuned by a learning algorithm while performing the control task. In this context, different components of the NMPC scheme in Eq. 4, including the stage and terminal costs, the controlled system's dynamical model, and state/input constraints, need to be parametrized. In the case of rich parametrization, the available RL framework can suitably control the system even if an accurate model of the system dynamics is unavailable.

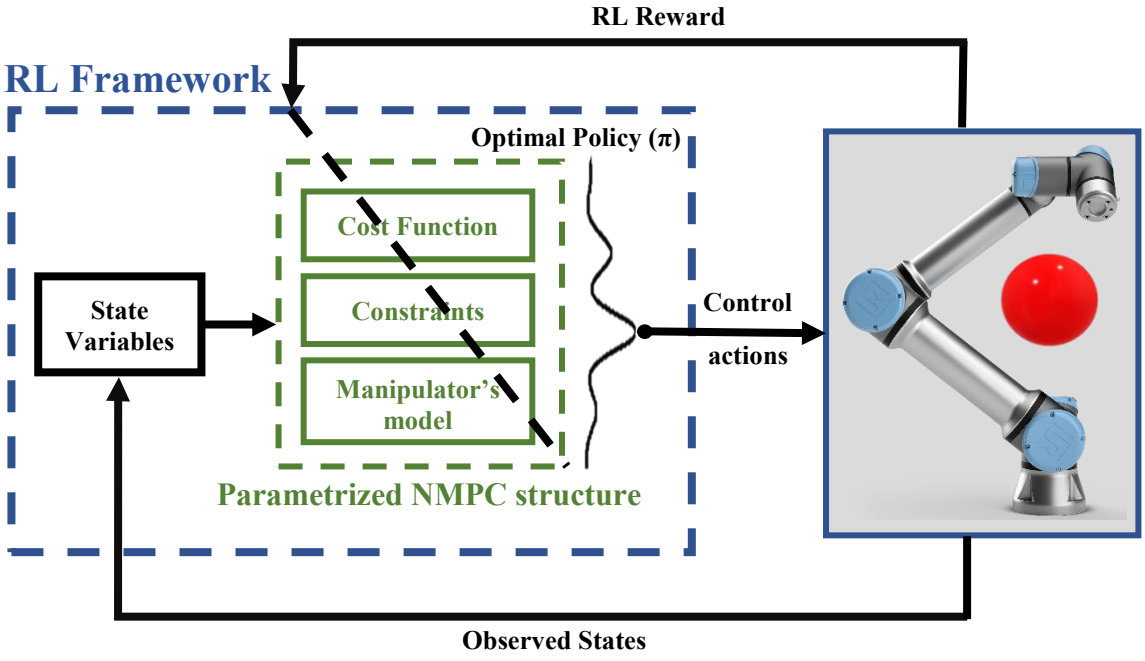


Fig. 2. A schematic of the NMPC-based RL framework. A parametrized NMPC is used as a function approximator for RL. NMPC parameters are tuned using a learning strategy according to the RL reward. The NMPC control outputs determine the optimal policy and actions in the RL framework.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

Eq. 5 shows how to formulate Eq. 4 for the setpoint control of the end-effector in the presence of obstacles.

$$Q_\theta(s,a) = \min_{x,u} \left[\gamma^N\left(X_N^T G_\theta^f X_N\right) + \eta_N^T \xi_N + \sum_{k=0}^{N-1} \gamma^k\left((X_k^T G_\theta X_k + U_k^T R_\theta U_k) + \eta^T \xi_k\right)\right] \quad (5a)$$

$$s.t. \quad X_k = [x_k^1, \ldots, x_k^6]^T, \quad U_k = [u_k^1, \ldots, u_k^6]^T \quad (5b)$$

$$X_{k+1} = f_\theta(X_k, U_k) \quad (5c)$$

$$O(X) + \theta_o \leq \xi_k, \quad \xi_{0,1,..,N} \geq 0 \quad (5d)$$

$$\dot{q}_{min}^{1,..N} \leq u_k^{1,..,N} \leq \dot{q}_{max}^{1,..N}, \quad x_{min}^{1,..N} \leq x_k^{1,..,N} \leq x_{max}^{1,..N} \quad (5e)$$

$$U_0 = a \quad (5f)$$

$Q_\theta(s,a)$ is the action-value function in the RL framework, calculated using the above MPC formulation. γ is a discount factor (standard MPC formulation in Eq. 4 doesn't include this factor), N is the prediction horizon, G_θ^f, G_θ, and R_θ are diagonal parametrized matrices, and $\xi_{0,1,..,N}$ are slack variables. Slack variables multiplied by η and η_N weights, for which large values are selected, are used to prevent the infeasibility of the problem because of applying the inequality mixed state-input constraints presented in Eq. 5d [27]. $O(X)$ is a function that determines the minimum distance between arm manipulator and obstacles. θ_o is an adjustable parameter that addresses the trade-off between setpoint control and obstacle avoidance. $O(X)$ and θ_o are presented in detail later in this section. Eq. 5e shows state variables' and control inputs' upper and lower bounds. Eq. 5f is a constraint demonstrating the relationship between RL actions (a) and MPC output (U_0).

The cost function of the NMPC structure in Eq. 5 directly addresses the setpoint control problem of the end-effector. With this formulation for terminal and stage costs, the aim is to minimize the end-effector pose control error and thus, reach the desired pose. In order to achieve the desired pose with the minimum rotation of the actuator joints, the velocity of the joints has also been included in the stage cost definition. Joints' velocities are the inputs to the kinematical model of the robot manipulator, which reflects the system's dynamics in the NMPC structure in Eq. 5c. To parametrize the cost function of the NMPC in Eq. 5a, R, G, and G^f have been considered diagonal parametrized matrices as in Eq. 6.

$$G^f = \begin{bmatrix} \theta_{G_f}^1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \theta_{G_f}^6 \end{bmatrix}, \; G = \begin{bmatrix} \theta_G^1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \theta_G^6 \end{bmatrix}, \; R = \begin{bmatrix} \theta_R^1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \theta_R^6 \end{bmatrix} \quad (6)$$

Collision avoidance is accomplished in two ways,

1. As an inequality constraint in the NMPC scheme presented in Eq. 5d [28].
2. As the RL framework's reward function.

One should note that collision avoidance may also be considered as an additional component in the stage cost definition in the NMPC scheme [29]. Nevertheless, using inequality constraint representation for this problem contributes to improving the parametrization of the NMPC structure, which, as previously mentioned, is a requirement for the available control framework. Collision avoidance is formulated as an inequality constraint using the distance between the robot manipulator and obstacles. The minimum distance between the robot manipulator components and obstacles must not fall below a safe threshold to prevent any collisions. Assuming a spherical obstacle with radius r and center at point P and robot links as cylinders with a diameter d^i, and P_1^i and P_2^i as their axis start and end points, the minimum distance between link i and the obstacle, d_o^i, is calculated as illustrated in Eq. 7 (see Fig. 3).

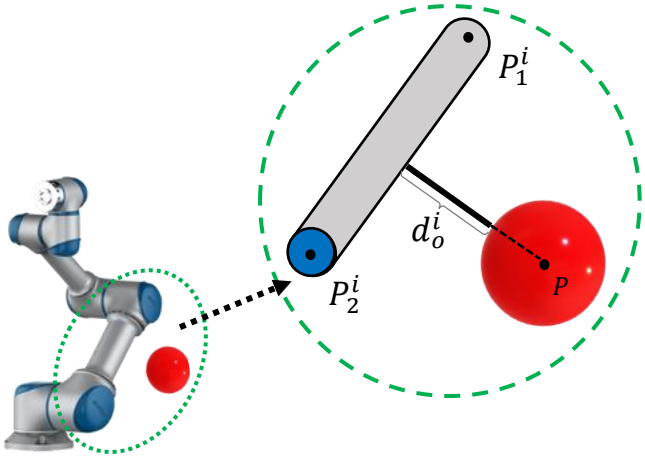


Fig. 3. The minimum distance (d_o^i) between a cylindrical link of the manipulator and a spherical obstacle in the robot vicinity.

$$d_o^i = \frac{\left|(P - P_1^i) \times (P - P_2^i)\right|}{\left|P_1^i - P_2^i\right|} - \frac{d^i}{2} - r \quad (7)$$

d_o^i with the lowest value would be the minimum distance between the robot manipulator and the obstacle. This calculated distance is referred to as $O(X)$ in Eq. 5d. P_1 and P_2 are determined by solving the manipulator's FK model.

The inequality constraint, Eq. 5d, is parametrized using θ_o. This variable determines the trade-off between obstacle avoidance and setpoint control. Increasing it enhances the distance between the robot manipulator and the obstacle, leading to safer manipulator movements. However, choosing too large values for θ_o affects the setpoint control problem and reduces the control loop's efficiency in reaching the desired pose. In the proposed control framework, RL is responsible for selecting appropriate values for θ_o.

To address the collision avoidance, the RL reward function is defined as follows [28],

$$L(X) = max\left(0, d_{tr} - O(X)\right) \quad (8)$$

where, $O(X)$ represents the minimum distance computed using the above explanations and d_{tr} is the safe threshold between the manipulator and the obstacle. It should be noted that decreasing the minimum distance between the

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

manipulator and the obstacle decreases the value of $O(X)$. According to Eq. 8, if the manipulator approaches the obstacle close enough to make $d_{tr} - O(X)$ positive, a penalty will be applied to the RL stage cost. Therefore, to achieve the optimal reward function, RL seeks to keep continually $d_{tr} - O(X)$ negative, thereby maintaining the minimum distance between the robot manipulator and obstacle greater than d_{tr}. One should note that based on the formulation presented in Eq. 8, the learning algorithm aims at minimizing the available RL reward function by tuning the NMPC parameters in order to reach the optimal solution for the control framework.

As part of the NMPC scheme, the manipulator's model, i.e., its kinematical model, should also be parametrized. For this aim, bias terms as variable parameters that RL will adjust are added to the physical parameters of the model, that is, the length of manipulator links. Thus, the length of each link, l_i, is replaced with $l_i + \theta_m^i$ in which θ_m^i is the tunable parameter.

As the control problem is being solved, RL adjusts the parameters of the NMPC scheme. This is accomplished by using a learning algorithm. Q-Learning is a model-free RL method that seeks to find the best possible action respecting the system's current state. Q-Learning uses the Temporal Difference method to tune the learning parameters according to the following update rule,

$$\delta_k = L(s_k, a_k) + \gamma V_\theta(s_{k+1}) - Q_\theta(s_k, a_k) \quad (9a)$$

$$\Delta\theta = \lambda \delta_k \nabla_\theta Q_\theta(s_k, a_k) \quad (9b)$$

$$\theta_{new} = \theta_{old} + \Delta\theta \quad (9c)$$

where s and a are state and action in the RL framework, and λ is the learning rate. The sensitivity term $\nabla Q_\theta(s_k, a_k)$ is required to calculate the Temporal Difference error using Eq. 9a. It is shown in [24] that,

$$\nabla_\theta Q_\theta(s, a) = \nabla_\theta \mathcal{L}_\theta(s, \mathcal{Y}^*) \quad (10)$$

where $\mathcal{L}$ is the Lagrange function formulation of the MPC scheme in Eq. 5 gathering the cost function and equality and inequality constraints. $\mathcal{Y}^*$ is the solution to Eq. 5.

The action-value function in Eq. 9a is calculated using the NMPC scheme presented in Eq. 5. One should note that by ignoring constraint Eq. 5f, the NMPC scheme in Eq. 5a will determine the value function ($V_\theta(s)$), and the deterministic policy can also be obtained as follows [24],

$$\pi_\theta(s) = U_0 = [\dot{q}_0^1, \dots, \dot{q}_0^6] \quad (11)$$

where U_0 is the first element of the vector U as the solution of the NMPC scheme (when Eq. 5f is ignored). It could be proved [24] that the value and action-value functions and the deterministic policy calculated using the above equations will satisfy the Bellman equations.

An epsilon-greedy policy is used to apply the exploration-exploitation trade-off in the available RL framework. For this aim, 90% of the generated actions are directly regarded as control signals. The remaining 10% are replaced with random values between the lower and upper bounds of the control inputs presented in Eq. 5e.

IV. Simulation Results and Discussion

A. Obstacle avoidance motion planning simulation

In this sub-section, the performance of the proposed control framework has been investigated through numerical simulations. To accomplish this, a general scenario of the end-effector pose control in the presence of static obstacles has been studied. The scenario includes moving the end-effector from an arbitrary initial position and orientation to reach the desired pose. A static spherical obstacle has also been considered in an arbitrary point in the manipulator's workspace between these two points. Table 1 shows the D-H parameters of the modeled robot manipulator.

TABLE I. D-H PARAMETERS OF THE MODELED ROBOT MANIPULATOR

Link	q_i	α_i	a_i	d_i
1	q_1	$\pi/2$	0	0.0892
2	q_2	0	0.425	0
3	q_3	0	0.392	0
4	q_4	$\pi/2$	0	0.1093
5	q_5	$\pi/2$	0	0.09475
6	q_6	0	0	0.0825

To parametrize the manipulator's model in Eq. 5c, links´ lengths (a_2, a_3, d_1, d_4, d_5, d_6 in Table 1) are biased using adjustable parameters θ_m^i ($i = 1, .., 6$). In addition to these six parameters, the diagonal elements of $G, R,$ and G^f matrices in NMPC stage and terminal cost functions presented in Eq. 5a ($\theta_G^i, \theta_R^i, \theta_{G^f}^i$, where $i = 1, \dots 6$), and θ_o presented in Eq. 5d are also tuned using RL. Thus, in total, 25 parameters are adjusted at each time step. Table 2 illustrates the initial values of the adjustable and fixed parameters of the control scheme.

TABLE II. PARAMETER VALUES IN THE CONTROL SCHEME

Parameter	Initial value	Parameter	Initial value
$\boldsymbol{\theta_G^i}$ $\boldsymbol{(i = 1, .., 6)}$	10	$\boldsymbol{\gamma}$	0.9
$\boldsymbol{\theta_R^i}$ $\boldsymbol{(i = 1, .., 6)}$	5	$\boldsymbol{d}$ **(Links' diameter)**	0.1
$\boldsymbol{\theta_{G^f}^i}$ $\boldsymbol{(i = 1, .., 6)}$	20	$\boldsymbol{r}$ **(Obstacle's radius)**	0.05
$\boldsymbol{\theta_m^i}$ $\boldsymbol{(i = 1, .., 6)}$	0.1	$\boldsymbol{\eta}$, $\boldsymbol{\eta_N}$	15
$\boldsymbol{\theta_o}$	0.1	$\boldsymbol{\xi}$, $\boldsymbol{\xi_N}$	22
$\boldsymbol{N}$	15	$\boldsymbol{d_{tr}}$	0.1

Joints have a maximum angular displacement of 2π rad and velocities of up to π rad/s. In order to be able to keep track of the joint angles and consider joint constraints, the state variables in Eq. 5b are extended as,

$$X_k = [x_k^1, \dots, x_k^6, q_k^1, .., q_k^6]^T \quad (12)$$

Thus, G and G^f matrices are replaced with,

$$G^f = \begin{bmatrix} G_{6\times6}^f & 0_{6\times6} \\ 0_{6\times6} & 0_{6\times6} \end{bmatrix}_{12\times12}, \quad (13)$$

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

$$G = \begin{bmatrix} G_{6\times6} & 0_{6\times6} \\ 0_{6\times6} & 0_{6\times6} \end{bmatrix}_{12\times12}$$

Considering the relationship between $q_{k+1}^{i}$and q_k^{i} as,

$$q_{k+1}^{i} = q_k^{i} + \Delta\tau u_k^{i} \quad (i = 1, \dots 6) \tag{14}$$

where $\Delta\tau$ denotes the control loop's time step, which has been taken as 0.05 sec.
The system model presented in Eq. 5c will also be modified as below,

$$X_{k+1} = [f_\theta(X_k, U_k)\ ,\ \Delta\tau u_k^1 + q_k^1\ ,\dots,\ \Delta\tau u_k^6 + q_k^6]^T \tag{15}$$

As explained in section I, Eq. 3 will be used as $f_\theta(X_k, U_k)$.

Fig. 4 shows the end-effector's trajectory when the controller is applied to the robot manipulator model in the presence of the obstacle.

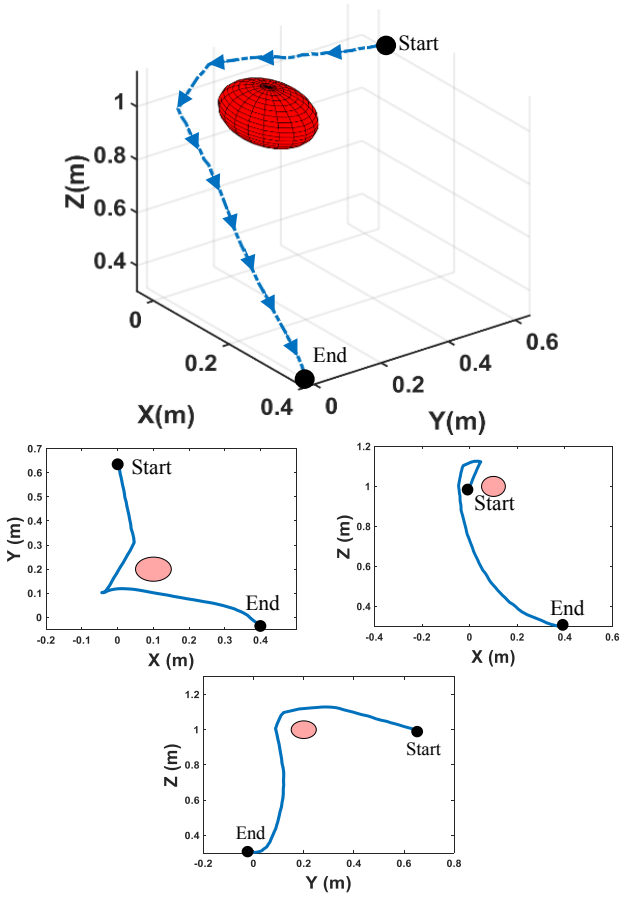


Fig. 4. The end-effector trajectory in 3D and from XY – XZ – YZ plane views. The controller moves the end-effector from the start point with an arbitrary initial condition to reach the end point with desired pose in a collision-free path.

Following are the initial and desired pose vectors of the end-effector in the simulation scenarios,

$$X_{init} = [0, 0.65, 1, 0, 0, 0] \tag{16a}$$

$$X_{desired} = \left[0.4\ , -0.05\ , 0.3\ , \frac{3\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{2}\right] \tag{16b}$$

As Fig. 4 illustrates, the available controller has successfully moved the end-effector to reach the desired pose in a collision-free path. Fig. 5 shows the trajectories of position and the orientation of the end-effector.

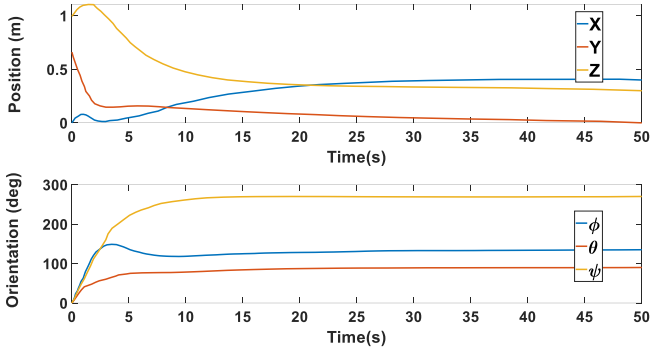


Fig. 5. The end-effector's position and orientation trajectories. The controller moves the end-effector to reach the desired pose as given in Eq. 16b.

The control actions, i.e., joint velocities as well as the angular displacement of each joint, are shown in Fig. 6. One should note from figures 5 and 6 that joints' displacement and velocity constraints have been satisfied during the simulation.

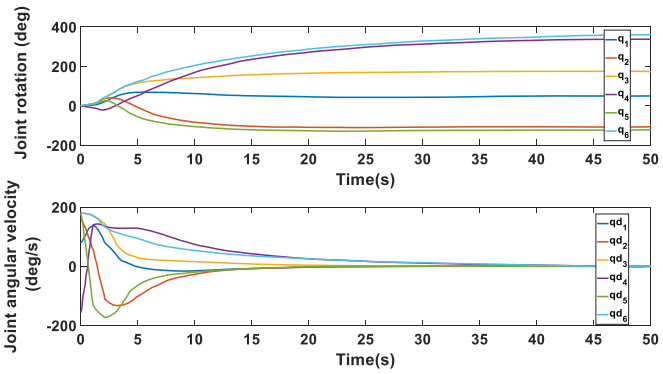


Fig. 6. Joints' angular velocities and rotation trajectories. All kinematical constraints are satisfied during the movements of the end-effector.

Fig. 7 shows how NMPC parameters have converged over the simulation time. For simplicity's sake, only the first parameter of NMPC cost function matrices, model parameters, and collision avoidance parameter, have been illustrated.

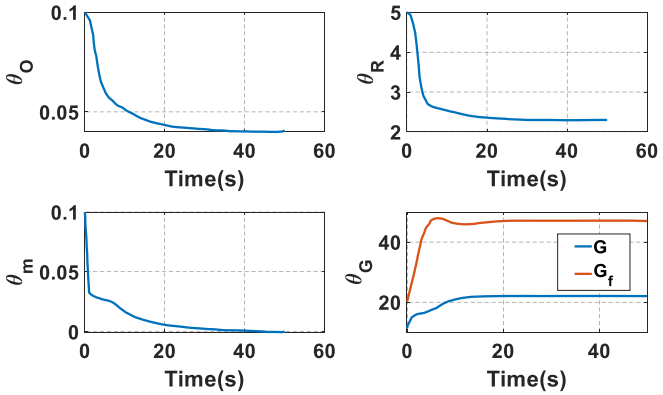


Fig. 7. Variations of some selected NMPC parameters during one simulation. θ_o, the obstacle avoidance parameter, is decreased over the simulation time in order to address the tradeoff between setpoint control and obstacle avoidance. θ_m, the manipulator model's parameter is converged to 0. θ_G and θ_{Gf} are increased during the simulation while θ_R is decreased as the simulation proceeds.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

The learning process is performed at each time step based on Eq. 9. As the learning trajectories demonstrate, G and G^f matrices parameters are relatively small at the beginning of the simulation, and their values increase over the learning steps. In contrast with these two matrices, the R matrix parameters' values decrease as the learning progresses. It means that the weighting matrix of the control actions in the stage cost function has been initialized with large values. All parameters in each NMPC cost function matrix, i.e., G, G^f, and R show the same behavior but with different increase/decrease change rates. The manipulator's model parameters demonstrate a sharp decrease at the begging of the simulation, and they converge to values close to zero very fast. This behavior seems reasonable since an exact kinematics model of the manipulator is available, and the parameterized model has been initialized well with the accurate values for links' lengths, etc. When it comes to the obstacle avoidance parameter, θ_o, one should note that it has been decreased over the simulation time. It means that this parameter has been initialized conservatively. Indeed, the initial value of θ_o does not properly address the trade-off between safety (collision avoidance) and efficiency (reaching the desired pose). RL tries to optimize this parameter concerning the threshold distance, d_{tr}, by reducing its value over the learning process.

B. Analysis of the controller's learning capablity

In this sub-section, the learning capability of the controller has been examined. To understand how the learning capability affects the efficiency of the controller, the changes of the closed control loop performance have been studied when increasing the learning steps. The results mentioned above come from only one simulation of the motion planning scenario. Although the NMPC parameters have been tuned during this simulation, enough learning epochs should be run to reach their desired values leading to the RL's optimal policy. To accomplish this, the simulation has been repeated several times. The NMPC parameters at each learning step are initialized with their values obtained at the end of the previous step. Fig. 8 illustrates the performance of the closed control loop, in terms of the RL stage cost (reward function) over the learning steps. As it could be understood, the RL stage cost decreases until it converges after eighteen steps with a 21% improvement, meaning that the near optimal policy is found after eighteen learning steps. The number of learning steps to reach the optimal policy may vary according to the complexity of the simulation scenario, e.g., the end-effector's initial and desired poses and the number of obstacles and their positions. The results of increasing the learning steps indicate that due to having the real-time learning capability, the efficiency of the closed control loop is improved as the simulation proceeds. This improvement in the control loop's performance due to the learning capability is the main advantage of the NMPC-based RL control framework compared to non-learning-based controllers like traditional NMPCs. One should note that in contrast with the available control algorithm, traditional control frameworks without learning features do not show overall performance improvements over time since they have non-adaptive structures with fixed parameters. The available controller is a good alternative for addressing the safety-efficiency trade-off in control problems. In the collision-free motion planning for robot manipulators studied in this paper, addressing the collision avoidance leads to a decrease in the control loop's performance in reaching the desired pose. Using the NMPC-based RL controller can increase the efficiency of the control loop and, to a large extent, help achieve a suitable trade-off between safety and efficiency.

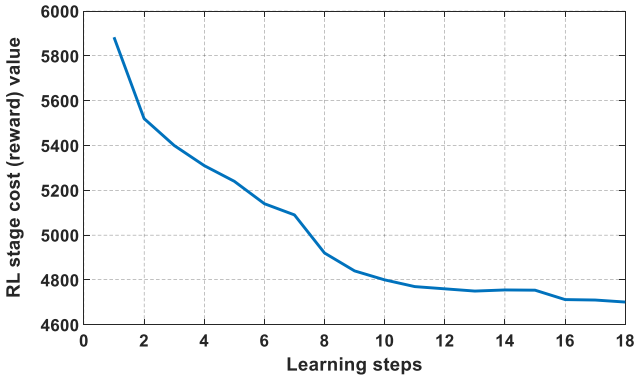


Fig. 8. Closed-loop performance in terms of RL stage cost value over the learning steps. After eighteen steps, the RL stage cost changes from 5873 to 4721, converges to the optimal value with a 21% improvement, meaning that the NMPC parameters have reached their near optimal values.

V. CONCLUSION

In this study, an NMPC-based RL control framework has been proposed for controlling the end-effector's pose in a 6-DoF robot manipulator in the presence of static obstacles. The controller's performance has been evaluated in a motion planning scenario through numerical simulations. Results indicate that the proposed controller can effectively move the end-effector from an initial pose to reach the desired pose in a collision-free path. A parametrized NMPC structure has been used in the available control scheme as value/action-value function approximator of the RL framework. The parameters of the NMPC are updated in real-time using the Temporal Difference method aimed at improving the closed control loop performance. The obtained results confirm that the controller's learning capability helps improve the control loop's performance up to 21% over time.

As the first step, in this paper the efficiency of the NMPC-based RL control structure has been studied through the simulation of a simple motion planning scenario including only one static obstacle. For future works, the controller's performance will be evaluated for the motion planning of redundant arm manipulators in the presence of multiple dynamic obstacles like humans through experimental implementations in real-world human-robot interaction scenarios.

ACKNOWLEDGMENT

This work is partially supported by The Research Council of Norway as a part of the Vulnerability in the Robot Society (VIROS) project, under grant agreement 288285, Predictive and Intuitive Robot Companion (PIRC) project

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.

under grant agreement 312333 and through its Centers of Excellence scheme, RITMO with the project no. 262762.

REFERENCES

[1] Lewis, F.L., D.M. Dawson, and C.T. Abdallah, *Robot manipulator control: theory and practice*. 2003: CRC Press.

[2] Mosher, R.S., *Industrial manipulators.* Scientific American, 1964. **211**(4): p. 88-97.

[3] Sung, G.T. and I.S. Gill, *Robotic laparoscopic surgery: a comparison of the da Vinci and Zeus systems.* Urology, 2001. **58**(6): p. 893-898.

[4] Pages, J., L. Marchionni, and F. Ferro. *Tiago: the modular robot that adapts to different research needs*. in *International workshop on robot modularity, IROS*. 2016.

[5] Lasota, P.A., T. Fong, and J.A. Shah, *A survey of methods for safe human-robot interaction*. Vol. 104. 2017: Now Publishers Delft, The Netherlands.

[6] Lacevic, B., P. Rocco, and A.M. Zanchettin, *Safety assessment and control of robotic manipulators using danger field.* IEEE Transactions on Robotics, 2013. **29**(5): p. 1257-1270.

[7] Mišeikis, J., et al. *Multi 3D camera mapping for predictive and reflexive robot manipulator trajectory estimation*. in *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*. 2016.

[8] Gerdts, M., et al., *Path planning and collision avoidance for robots.* Numerical Algebra, Control & Optimization, 2012. **2**(3): p. 437.

[9] Xu, X., et al., *A novel non-collision trajectory planning algorithm based on velocity potential field for robotic manipulator.* International Journal of Advanced Robotic Systems, 2018. **15**(4): p. 1729881418787075.

[10] Zardykhan, D., et al. *collision preventing phase-progress control for velocity adaptation in human-robot collaboration*. in *2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids)*. 2019. IEEE.

[11] Sangiovanni, B., et al. *Deep reinforcement learning for collision avoidance of robotic manipulators*. in *2018 European Control Conference (ECC)*. 2018. IEEE.

[12] Baselizadeh, A., D. Saplacan, and J. Tørresen. *Adaptive Real-time Learning-based Neuro-Fuzzy Control of Robot Manipulators*. in *2021 9th International Conference on Control, Mechatronics and Automation (ICCMA)*. 2021.

[13] Camacho, E.F. and C.B. Alba, *Model predictive control*. 2013: Springer science & business media.

[14] Carron, A., et al., *Data-Driven Model Predictive Control for Trajectory Tracking With a Robotic Arm.* IEEE Robotics and Automation Letters, 2019. **4**(4): p. 3758-3765.

[15] Krämer, M., et al., *Model predictive control of a collaborative manipulator considering dynamic obstacles.* Optimal Control Applications and Methods, 2020. **41**(4): p. 1211-1232.

[16] Zube, A. *Cartesian nonlinear model predictive control of redundant manipulators considering obstacles*. in *2015 IEEE International Conference on Industrial Technology (ICIT)*. 2015. IEEE.

[17] Capcha, M.A., W. Ipanaqué, and R.D. Keyser. *Comparison of model-based and non-model-based strategies for nonlinear control of a three-tank system*. in *2017 22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2017.

[18] Berkenkamp, F., et al., *Safe model-based reinforcement learning with stability guarantees.* Advances in neural information processing systems, 2017. **30**.

[19] Brunke, L., et al., *Safe learning in robotics: From learning-based control to safe reinforcement learning.* Annual Review of Control, Robotics, and Autonomous Systems, 2021. **5**.

[20] Hewing, L., et al., *Learning-based model predictive control: Toward safe learning in control.* Annual Review of Control, Robotics, and Autonomous Systems, 2020. **3**: p. 269-296.

[21] Recht, B., *A Tour of Reinforcement Learning: The View from Continuous Control.* Annual Review of Control, Robotics, and Autonomous Systems, 2019. **2**(1): p. 253-279.

[22] Hester, T., et al. *Deep q-learning from demonstrations*. in *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018.

[23] Watkins, C.J. and P. Dayan, *Q-learning.* Machine learning, 1992. **8**(3): p. 279-292.

[24] Gros, S. and M. Zanon, *Data-Driven Economic NMPC Using Reinforcement Learning.* IEEE Transactions on Automatic Control, 2020. **65**(2): p. 636-648.

[25] Kebria, P.M., et al. *Kinematic and dynamic modelling of UR5 manipulator*. in *2016 IEEE international conference on systems, man, and cybernetics (SMC)*. 2016. IEEE.

[26] Sutton, R.S. and A.G. Barto, *Reinforcement learning: An introduction*. 2018: MIT press.

[27] Tapia, R.A., *Role of slack variables in quasi-newton methods for constrained optimization*. 1979, Rice Univ., Houston, TX (USA). Dept. of Mathematical Sciences.

[28] Cai, W., et al., *MPC-based Reinforcement Learning for a Simplified Freight Mission of Autonomous Surface Vehicles.* arXiv preprint arXiv:2106.08634, 2021.

[29] Jasour, A.M. and M. Farrokhi. *Path tracking and obstacle avoidance for redundant robotic arms using fuzzy NMPC*. in *2009 American Control Conference*. 2009. IEEE.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on November 05,2026 at 18:26:53 UTC from IEEE Xplore. Restrictions apply.